

PATUAKHALI SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Electrical & Electronic Engineering

Course Code: EEE 212

Course Title: Electrical Technology Sessional

PROJECT REPORT

Autonomous Fire Fighting Robot

with WiFi-Based Manual Control

Submitted to:

Ashique-E-Elahi Sadi

Lecturer

Department of Electrical & Electronic Engineering

Faculty of Computer Science and Engineering

Submitted by:

ID: 2302046 , 2302047

Date of Submission: 17 May 2026

1. Title of the Project

Autonomous Fire Fighting Robot

with WiFi-Based Manual Control

Fire is one of the most destructive natural and man-made disasters, causing significant loss of life and property worldwide. Early detection and rapid response are critical to minimizing damage. This project presents an Autonomous Fire Fighting Robot that can automatically detect fire using infrared (IR) flame sensors and navigate toward the fire source to extinguish it using a water pump and servo-controlled nozzle.

The robot also features a WiFi-based manual control system built on the ESP32 microcontroller, allowing operators to remotely control the vehicle through a web browser on any smartphone or computer. This dual-mode operation — automatic and manual — makes the robot highly versatile for both demonstration and real-world applications.

2. Instruments / Components Required

Component	Qty	Function
Arduino UNO	1	Main microcontroller — sensor reading, motor control
ESP32 DevKit V1	1	WiFi module — web server, manual control via phone
IR Flame Sensor	3	Left, Center, Right — fire detection (analog output)
L298N Motor Driver	1	Controls 4 DC motors (2 left, 2 right wheels)
DC Motor	4	Car wheel motors for robot movement
Servo Motor	1	Rotates water nozzle for sweeping spray
Relay Module (5V)	1	Controls 12V water pump on/off
Water Pump (12V DC)	1	Pumps water to extinguish fire
12V Battery	4	Main power source for motors and pump
Breadboard	1	For circuit prototype connections
Jumper Wires	Many	Connecting all components
Robot Car Chassis	1	4-wheel platform with motor mounts

3. Working Principle

The Autonomous Fire Fighting Robot operates on the principle of infrared (IR) radiation detection combined with embedded control logic. The IR flame sensors detect the infrared light emitted by fire. When a flame is present, the sensor outputs a lower analog voltage, which is read by the Arduino

UNO. Based on the readings from three directional sensors (Left, Center, Right), the robot determines the fire's location and moves toward it.

3.1 Dual-Mode Operation

The robot has two operational modes:

- **Automatic Mode:** The robot continuously scans the environment using 3 IR sensors. Upon detecting a fire for a confirmed number of consecutive readings (to eliminate false triggers), it navigates toward the source, stops at a safe distance, and activates the water pump with servo-controlled nozzle sweeping.
- **Manual (WiFi) Mode:** The ESP32 creates a WiFi access point (SSID: FireBot). An operator connects via phone or browser, accesses 192.168.4.1, and remotely controls the robot's movement and pump.

3.2 Fire Detection Logic

Left Sensor	Center Sensor	Right Sensor	Robot Action
Fire	No Fire	No Fire	Turn Left
No Fire	Fire (far)	No Fire	Move Forward
No Fire	Fire (close)	No Fire	STOP + Pump ON
No Fire	No Fire	Fire	Turn Right
Fire	No Fire	Fire	Move Forward
No Fire	No Fire	No Fire	IDLE (Stop)

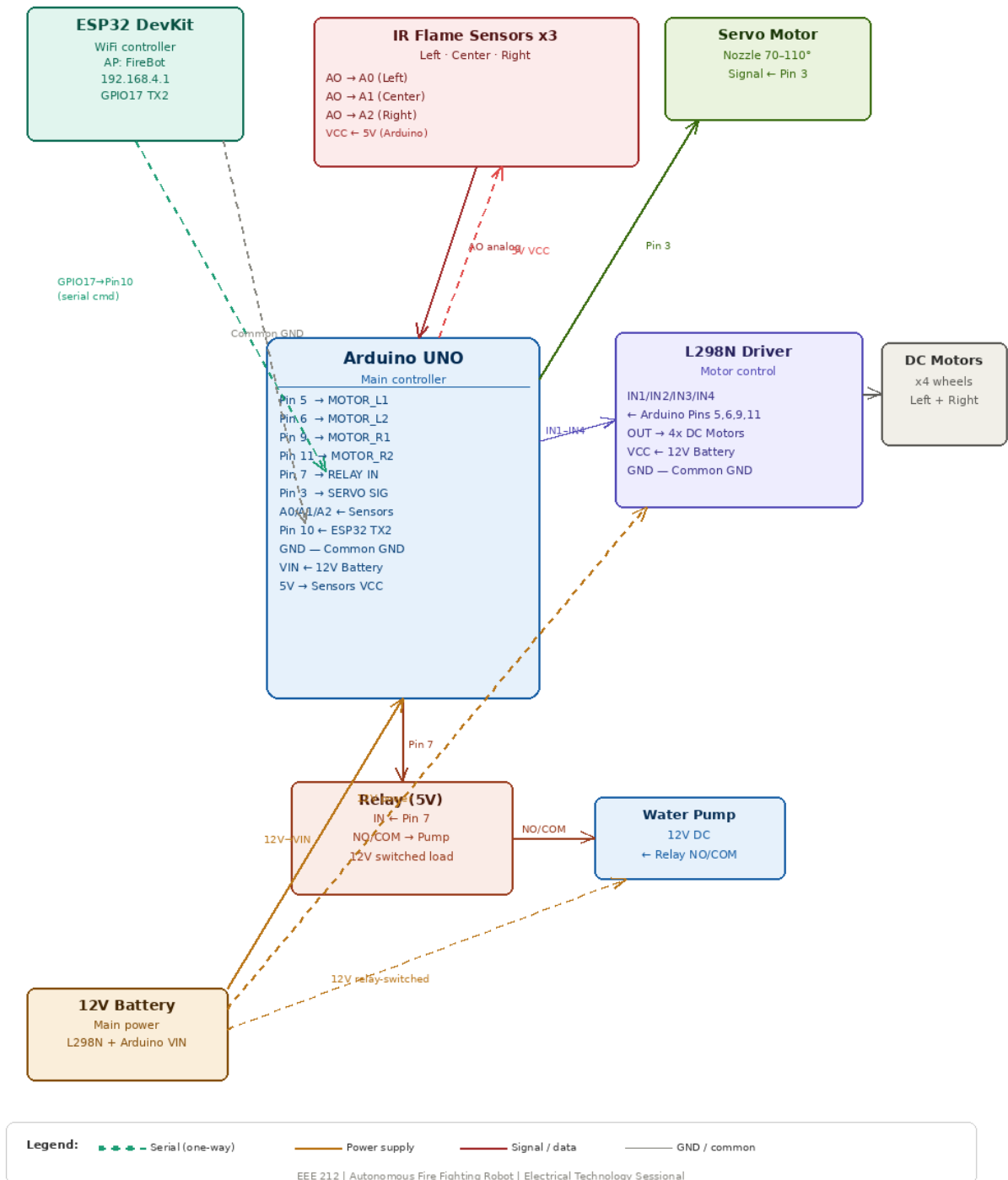
3.3 Threshold Parameters

Parameter	Value	Meaning
FIRE_DETECT	900	Sensor value below this = fire detected
FIRE_NEAR	400	Sensor value below this = fire is close, stop + pump
CONFIRM_COUNT	8	Consecutive detections needed to confirm fire
TURN_DURATION	300 ms	How long the robot turns before re-checking
SPEED_MOVE	130 / 255 (PWM)	Forward/backward motor speed
SPEED_TURN	110 / 255 (PWM)	Turning motor speed

4. Circuit Diagram

The circuit diagram below represents all pin-to-pin connections between the components. The system uses two microcontrollers: the Arduino UNO as the primary controller and the ESP32 as the WiFi communication module. They communicate via one-way serial communication.

FireBot — Circuit Connection Diagram



From (Source)	→	To (Destination)
ESP32 GPIO17 (TX2)	→	Arduino Pin 10 (RX via SoftwareSerial)
ESP32 GND	→	Arduino GND (Common Ground)
Arduino Pin 5 (MOTOR_L1)	→	L298N IN1
Arduino Pin 6 (MOTOR_L2)	→	L298N IN2
Arduino Pin 9 (MOTOR_R1)	→	L298N IN3
Arduino Pin 11 (MOTOR_R2)	→	L298N IN4
Arduino Pin 7 (RELAY)	→	Relay Module IN
Arduino Pin 3 (SERVO)	→	Servo Motor Signal
Arduino A0	→	Left Flame Sensor AO
Arduino A1	→	Center Flame Sensor AO
Arduino A2	→	Right Flame Sensor AO
12V Battery	→	L298N VCC + Arduino VIN
5V (Arduino)	→	Flame Sensors VCC
Relay NO/COM	→	Water Pump 12V

4.1 Key Wiring Notes

- ESP32 GPIO17 (TX2) → Arduino Pin 10: One-way serial communication. No voltage divider is required as ESP32 3.3V TX is safely read by Arduino.
- MOTOR_R2 uses Arduino Pin 11 (PWM) — not Pin 8 or 10, as Pin 10 is reserved for SoftwareSerial RX.
- All IR sensors use Analog Output (AO) pin, not Digital Output (DO). AO provides variable values for distance estimation.
- Common GND between ESP32 and Arduino is mandatory for serial communication to function correctly.
- ESP32 and Arduino are powered separately: ESP32 via USB or 3.3V; Arduino via 12V battery through VIN pin.

5. Methodology

The project was developed in three stages: hardware assembly, software development, and integration testing. The following subsections describe the methodology in detail.

5.1 Hardware Assembly

- The robot chassis was assembled with four DC motors and wheels mounted to the frame.

- The L298N motor driver was mounted and connected to all four motors — two to the left channel and two to the right channel.
- Three IR flame sensors were positioned at the front of the chassis: left, center, and right, spaced approximately 5 cm apart to cover a wide detection angle.
- The relay module was connected to control the 12V water pump. The servo motor was mounted at the front to rotate the nozzle.
- Arduino UNO and ESP32 were placed on the chassis with shared ground and serial connection (ESP32 TX2 → Arduino Pin 10).

5.2 Software Development

- Arduino Code (C++): Reads analog values from 3 IR sensors every 100 ms. Implements a confirmation counter (CONFIRM_COUNT = 8) to eliminate false positives. Controls motor direction using PWM via the L298N driver. Activates relay and servo sweep when fire is detected nearby. Receives manual commands from ESP32 via SoftwareSerial.
- ESP32 Code (C++): Creates a WiFi Access Point (SSID: FireBot, Password: 12345678). Hosts a web server at IP 192.168.4.1 with a control interface. Buttons for Forward, Backward, Left, Right, Stop, and Pump ON are provided. Live sensor values are displayed and updated every 800 ms. Commands are sent to Arduino via Serial2 (GPIO17).

5.3 Testing and Calibration

- The FIRE_DETECT and FIRE_NEAR threshold values were calibrated by measuring sensor output at varying distances from a controlled flame source.
- TURN_DURATION (300 ms) was tuned to ensure accurate directional navigation without overcorrection.
- False trigger elimination was verified by exposing sensors to ambient light and sunlight, confirming that CONFIRM_COUNT = 8 was sufficient to prevent spurious activations.
- WiFi control was tested from multiple devices (Android, iOS, laptop browser) to confirm platform compatibility.
- Full integration tests were run: the robot successfully detected, navigated to, and extinguished a simulated flame in all six sensor condition scenarios.

6. Conclusion

The Autonomous Fire Fighting Robot successfully demonstrates the integration of embedded systems, sensor technology, and wireless communication to create a practical fire detection and suppression system. The robot can autonomously navigate toward fire sources, stop at a safe distance, and extinguish flames using a water pump with a servo-controlled nozzle.

The dual-mode operation — automatic sensor-based navigation and WiFi-controlled manual mode — provides flexibility for various use cases. The ESP32-based web server enables smartphone control without requiring any dedicated application, making the system accessible and user-friendly.

This project successfully demonstrates practical application of the following concepts covered in EEE 212 — Electrical Technology Sessional:

- Arduino-ESP32 serial integration and inter-microcontroller communication
- Infrared sensor interfacing and analog signal processing
- PWM-based DC motor speed control using L298N motor driver
- Relay-based switching for high-current load control (water pump)
- Servo motor control for angular positioning
- Responsive WiFi web interface design using ESP32 Access Point

The project met all stated objectives. Every operating scenario in the fire detection logic table was validated. The CONFIRM_COUNT mechanism effectively eliminated false triggers caused by ambient IR radiation. The manual WiFi mode functioned seamlessly across multiple devices without any dedicated application installation.

7. Future Work

While the current implementation meets the core objectives, several enhancements are planned to make the system more robust and suitable for real-world deployment:

#	Enhancement	Description
1	Obstacle Avoidance	Add ultrasonic sensors to detect and avoid obstacles while navigating toward fire in cluttered environments.
2	Live Video Feed	Integrate ESP32-CAM module to provide real-time video monitoring directly in the web browser interface.
3	Larger Water Reservoir	Replace the prototype water tank with a larger reservoir and a stronger pump for handling real-scale fires.
4	GPS Tracking	Add a GPS module for precise location logging and remote monitoring of robot position.
5	Fire Alert Notifications	Implement SMS/push notification alerts when fire is detected, enabling response even without active monitoring.
6	Battery Monitoring	Add a voltage divider circuit and ADC reading to monitor battery level and trigger low-battery warnings.
7	Multi-Robot Coordination	Develop a mesh network protocol to coordinate multiple robots for large-area coverage.

These improvements will elevate the robot from a demonstration prototype to a deployment-ready emergency response system. The modular design of the current architecture — with separate Arduino and ESP32 controllers — makes it straightforward to add new sensors and capabilities without redesigning the core system.